web service" or "stop the database." This is in stark contrast to the low-level docker run commands we've seen, where the context of what we're trying to achieve is harder to see.

Before we can get Compose to do our bidding, though, we first have to describe our application by creating a docker-compose.yml file. This doesn't replace the need for Dockerfiles—blueprints for creating containers—but it describes what images our app requires and how they are run in concert.

Here's an initial docker-compose.yml for our Rails project:

```
Line 1  version: '3'
2
3  services:
4
5    web:
6      build: .
7      ports:
8        - "3000:3000"
```

A Compose file always starts with a version number (line 1), which specifies the file format being used. This helps ensure that apps continue to run as expected as new features, or breaking changes, are added to Compose. We're using version 3—the latest major version at the time of writing.[1]

Next we have a collection called services (line 3), which is used to group the constituent parts of our application. Currently, our Rails app *is* our entire app, so we're declaring a single service—which we've chosen to call web—to represent it (line 5). We'll soon be adding another service in Chapter 5, Beyond the App: Adding Redis, on page 59.

Nested under web are its various configuration options (lines 6–8).

The first of these:

```
build: .
```

tells Compose where to find the Dockerfile it should use to build our image. The path we specify is relative to the docker-compose.yml file. In this case, it's in the same directory.

Next we come to lines 7–8:

```
ports:
  - "3000:3000"
```

1. https://docs.docker.com/compose/compose-file/compose-versioning/